

Unity ECS 기반 대규모 3D 인스턴스 렌더링 성능 분석 및 최적화



미오

202255658 김재식

202155593 이천서

202255568 안지홍

지도교수: 박영진

제출일: 2026년 9월 11일

부산대학교

목차

1	서론	1
1.1	연구 배경 및 기존 문제점	1
1.2	연구 목표	2
2	연구 배경	3
2.1	데이터 지향 설계와 ECS 기반 상태 처리	3
2.2	대규모 인스턴스 렌더링과 GPU 제출 구조	4
2.2.1	동적 인스턴스와 GPU 자원 관리	4
2.2.2	GPU 작업 집계와 Unity 렌더링 제출 구조	4
3	연구 내용	5
3.1	비교 구조의 재설계	5
3.2	시스템 구조 및 구현	6
3.2.1	ECS 스왑 시뮬레이션 및 애니메이션	6
3.2.2	RMI 렌더링 경로	6
3.2.3	BRG 렌더링 경로	7
3.2.4	통합 및 4분할 제출 구조	7
3.3	성능 측정 및 실험 설계	8
3.3.1	측정 지표와 수집 방식	8
3.3.2	실험 조건과 부하 구성	9
3.3.3	벤치마크 환경 및 통제 조건	10
3.3.4	자동 측정 및 분석 절차	11
4	연구 결과 분석 및 평가	12
4.1	프로젝트 수행 결과	12
4.1.1	개발 일정	12
4.1.2	구성원 역할 분담	12

4.2	성능 실험 결과 및 분석	12
4.2.1	렌더링 경로 및 제출 구조의 성능 비교	15
4.2.2	단계별 처리 시간과 해석	16
4.2.3	부하 조건에 따른 성능 변화	16
4.3	연구 성과	17
5	결론 및 향후 연구 방향	18
5.1	결론 및 기대효과	18
5.2	향후 연구 방향	18

제1장 서론

1.1 연구 배경 및 기존 문제점

실시간 게임과 시뮬레이션에서는 다수의 객체가 서로 다른 위치, 애니메이션 및 행동 상태를 유지하면서 동시에 가시화되는 대규모 동적 장면이 요구된다. 객체 수가 증가하면 화면에 처리해야 하는 기하 정보뿐 아니라 객체의 상태 갱신, 인스턴스 데이터 구성 및 전송, 렌더링 명령 제출에 필요한 비용도 함께 증가한다. 따라서 대규모 인스턴스 렌더링의 성능은 단순한 그래픽 처리량만으로 판단하기보다, 동적으로 갱신되는 객체 상태가 실제 렌더링 결과로 전달되는 전체 처리 과정의 관점에서 분석할 필요가 있다[1, 2].

대량 객체의 상태를 구조화하고 반복적으로 갱신하기 위한 구조로 Entity Component System(ECS)이 활용되고 있다[8]. 그러나 ECS에서 객체 상태를 갱신한 이후에도 이를 GPU가 처리할 수 있는 인스턴스 데이터와 렌더링 명령으로 구성하여 전달하는 과정이 필요하므로, 렌더링 경로와 제출 구조 역시 전체 시스템 성능을 결정하는 주요 설계 요소가 된다. Unity는 대규모 인스턴스를 처리할 수 있는 저수준 렌더링 방식으로 Graphics.RenderMeshIndirect(RMI)와 BatchRendererGroup(BRG)을 제공하며, 두 경로는 렌더링 데이터와 명령을 구성하고 제출하는 방식에서 차이를 가진다[7, 5]. 또한 동일한 렌더링 경로에서도 인스턴스를 하나의 단위로 통합하거나 여러 단위로 분할하여 제출함에 따라 명령 구조가 달라질 수 있다.

기존 연구에서는 데이터 지향 상태 처리, 대규모 군중 및 인스턴스 렌더링, GPU 기반 작업 집계와 같은 문제를 각각 다루어 왔다[8, 1, 3]. 그러나 동일한 Unity ECS 기반의 동적 상태와 렌더링 조건을 유지하면서 RMI와 BRG의 성능을 비교하고, 제출 구조의 영향을 함께 구분하여 분석한 정량적 자료는 제한적이다. 이에 동일한 시뮬레이션과 렌더링 자원을 사용하는 비교 환경을 구축하고, 렌더링 경로와 제출 구조가 대규모 동적 인스턴스 렌더링의 성능 특성에 미치는 영향을 체계적으로 분석할 필요가 있다.

1.2 연구 목표

본 프로젝트의 목표는 Unity ECS 기반의 대규모 동적 3D 인스턴스 시스템을 구축하고, 공통 ECS 시뮬레이션 로직과 상태 구성 및 렌더링 조건에서 RMI와 BRG의 두 렌더링 경로에 통합 제출과 4분할 제출을 교차 적용하여 비교 가능한 성능 측정 환경을 구성하는 것이다. 이를 위해 렌더링 경로와 제출 구조 이외의 주요 조건을 공통으로 유지하고, 각 조건을 반복적으로 실행하여 성능 차이를 정량적으로 확인할 수 있도록 시스템을 구성한다.

구축한 시스템을 이용하여 렌더링 경로에 따른 성능 차이, 통합 및 4분할 제출 구조의 효과, 그리고 부하 변화에 따른 프레임 성능과 처리 특성의 변화를 분석한다. 이를 통해 특정 렌더링 경로의 보편적인 우위를 제시하기보다, Unity ECS 기반 대규모 동적 인스턴스 시스템에서 부하 조건에 따라 렌더링 경로와 제출 구조를 선택하고 최적화 우선순위를 판단하기 위한 실증적 근거를 제시하는 것을 최종 목표로 한다.

제2장 연구 배경

2.1 데이터 지향 설계와 ECS 기반 상태 처리

데이터 지향 설계는 프로그램에서 반복적으로 처리되는 데이터의 배치와 접근 방식을 중심으로 구조를 구성하는 접근이다. 다수의 데이터를 연속적으로 처리할 때는 데이터의 저장 위치와 접근 순서가 메모리 지역성과 캐시 이용에 영향을 줄 수 있으며, 동일하거나 유사한 연산을 반복하는 대량 처리에서도 이러한 특성이 중요하게 작용한다. 객체 지향 구조와 데이터 지향 구조를 동일한 게임에 적용하여 비교한 연구에서는 실험에 사용된 데이터 지향 구현이 단일 스레드와 다중 스레드 조건에서 더 짧은 실행 시간과 낮은 캐시 미스율을 보였다[9]. 이 결과는 특정 구현과 실험 조건에서 얻어진 것이므로 데이터 지향 설계의 보편적인 성능 우위로 일반화하기보다는, 데이터의 배치와 접근 구조가 대량 상태 처리의 성능에 영향을 줄 수 있음을 보여주는 사례로 볼 수 있다.

ECS는 객체의 식별과 상태 데이터, 상태를 처리하는 로직을 분리하여 구성한다. 개별 엔티티의 상태는 컴포넌트 형태로 표현되고, 시스템은 필요한 컴포넌트 구성을 대상으로 상태를 갱신한다. 이러한 구조는 객체별로 상태와 동작을 결합하는 방식과 달리 동일한 종류의 데이터를 중심으로 처리 구조를 구성할 수 있다는 특징을 가진다. ECS 관련 연구를 체계적으로 조사한 결과에서도 성능, 모듈성 및 확장성과 관련된 장점이 다수 보고되었지만, 새로운 기술에 대한 긍정적 평가가 편향될 가능성이 함께 지적되었다 [8]. 따라서 ECS의 적용 자체를 성능 향상과 동일하게 간주하기보다 구체적인 데이터 구성과 실행 방식까지 함께 고려할 필요가 있다.

동일한 게임을 여러 ECS 프레임워크로 구현한 비교에서도 프레임률과 메모리 사용량에 차이가 나타났으며, 이는 구체적인 구현에 따라 성능 특성이 달라질 수 있음을 보여준다 [4]. 따라서 렌더링 구조를 평가할 때는 상태 처리 구조의 차이가 결과에 반영되지 않도록 비교 조건을 통제할 필요가 있다. 구체적인 ECS 상태 처리 구조와 실험 조건은 다음 장에서 설명한다.

2.2 대규모 인스턴스 렌더링과 GPU 제출 구조

2.2.1 동적 인스턴스와 GPU 자원 관리

대규모 동적 장면에서는 다수의 개체가 동일하거나 유사한 메시와 애니메이션 자원을 공유하면서도 서로 다른 위치, 방향 및 동작 상태를 유지해야 한다. 이러한 장면에서는 개체마다 동일한 렌더링 자원을 반복하여 생성하기보다 공통 자원을 재사용하고, 인스턴스별로 변화하는 상태를 별도로 관리하는 방식이 활용된다. 실시간 군중 렌더링 연구에서도 인스턴싱과 GPU 기반 애니메이션 처리 등 공통 자원의 재사용을 바탕으로 다수의 캐릭터를 효율적으로 가시화하기 위한 기법들이 다루어져 왔다 [1].

동적 상태와 공통 렌더링 자원을 분리하는 구조는 프레임마다 변화하는 데이터를 처리하는 데에도 활용된다. 대규모 군중 렌더링 사례에서는 시뮬레이션으로 생성된 위치와 방향을 인스턴스별 상태로 사용하고, 정점, 스켈레톤, 스키닝 가중치 및 애니메이션과 같이 여러 개체가 공유할 수 있는 데이터는 GPU 버퍼와 텍스처에 배치하여 재사용하였다[2]. 이와 같은 구조에서는 공통 자원을 유지하면서 프레임별 인스턴스 상태만 갱신할 수 있다. 따라서 대규모 동적 인스턴스의 성능은 기하 복잡도뿐 아니라 동적 상태를 GPU 데이터와 렌더링 작업으로 연결하는 방식의 영향도 받을 수 있다.

2.2.2 GPU 작업 집계와 Unity 렌더링 제출 구조

GPU를 이용한 대량 처리에서는 개별 작업의 연산량뿐 아니라 작업을 구성하고 제출하는 단위도 전체 처리 비용에 영향을 줄 수 있다. 여러 작업을 큰 단위로 집계하면 장면 자원의 저장, 렌더링, 데이터 적재 및 동기화에 반복되는 비용을 여러 요청에 걸쳐 상각할 수 있다. 다수의 독립 시뮬레이션 환경을 하나의 큰 배치로 처리한 연구에서도 이러한 작업 집계를 통해 반복되는 처리 비용을 전체 배치에 걸쳐 상각하는 구조가 사용되었다 [3]. 다만 해당 사례는 여러 독립 환경을 동시에 처리하는 구조이므로, 단일 장면에서 동일한 인스턴스 집합의 제출 단위를 변경하는 경우와 직접 동일한 조건으로 볼 수는 없다.

본 프로젝트가 비교하는 Unity의 RMI와 BRG도 다수의 인스턴스를 GPU에서 렌더링하기 위한 저수준 경로이지만, 렌더링 작업을 구성하고 엔진에 제공하는 방식에는 차이가 있다. RMI에서는 응용 프로그램이 간접 렌더링에 필요한 인스턴스 데이터와 실행 정보를 준비한 뒤 렌더링 작업을 제출한다[7]. BRG에서는 렌더링 자원을 사전에 등록하고 Unity의 렌더링 절차에 필요한 가시 인스턴스와 드로우 정보를 제공한다[5]. 따라서 두 경로는 모두 GPU의 인스턴스 데이터를 이용하지만, 렌더링 작업의 구성과 제출을 제어하는 구조에서는 차이를 가진다. 이러한 기술적 차이를 바탕으로 본 프로젝트에서 구성한 구체적인 비교 조건과 구현 방식은 다음 장에서 다룬다.

제3장 연구 내용

3.1 비교 구조의 재설계

착수 단계에서는 BRG를 중심으로 렌더링 전용 조건인 Render-only BRG와 ECS에서 갱신된 상태를 렌더링에 반영하는 ECS-fed BRG를 비교하고, 제출 구조의 차이도 함께 분석하는 방향으로 시스템을 설계하였다. 이를 통해 ECS와 렌더링 시스템의 연계 여부와 렌더링 제출 방식이 성능에 미치는 영향을 확인하고자 하였다.

그러나 프로토타입 구현과 비교 구조를 검토하는 과정에서 두 조건의 기능과 처리 부하를 동등하게 유지하기 어렵다는 문제가 확인되었다. Render-only BRG는 ECS에서 갱신되는 이동, 대열, 교전 및 애니메이션 상태를 실제 렌더링 과정에 반영하지 않는 반면, ECS-fed BRG는 해당 상태의 갱신과 렌더링 데이터 구성을 함께 수행한다. 따라서 두 조건의 성능 차이에는 렌더링 데이터 공급 방식뿐 아니라 시뮬레이션과 상태 처리 부하의 차이도 함께 포함되므로, 렌더링 구조 자체의 효과를 분리하여 해석하기 어렵다.

이에 데이터 공급 여부를 비교 요인에서 제외하고, 모든 조건에서 동일한 ECS 시뮬레이션 로직과 상태 구성 및 렌더링 자원을 사용하도록 비교 구조를 재설계하였다. 기존 ECS 군집 시뮬레이션과 GPU 스키닝 및 BRG 기반 구현은 공통 기반으로 유지하면서 RMI 렌더링 경로를 추가하였으며, 최종 비교 요인은 렌더링 경로와 제출 구조로 한정하였다. 착수 단계와 최종 설계의 주요 변경 내용은 표 3.1과 같다.

표 3.1: 착수 단계 대비 주요 비교 구조 변경

항목	착수 단계	최종 설계	변경 목적
렌더링 비교 요인	Render-only BRG / ECS-fed BRG	RMI / BRG	렌더링 경로 외 조건 차이 최소화
공통 시뮬레이션 조건	조건별 ECS 상태 반영 범위 상이	동일한 ECS 시뮬레이션 로직과 상태 구성 사용	비교 조건 간 시뮬레이션 부하 통제
제출 구조	유형별 통합 / 분할 Draw List	통합 제출 / 4분할 제출	제출 단위 변화의 영향 분리

3.2 시스템 구조 및 구현

최종 시스템은 모든 비교 조건에서 동일한 ECS 시뮬레이션 로직과 상태 구성을 사용하고, 여기에서 생성된 인스턴스 상태를 공통 렌더링 입력으로 사용하도록 구성하였다. 렌더링 단계에서는 RMI와 BRG의 요구 구조에 맞게 처리 경로를 분리하고, 두 경로에 동일한 기준의 통합 및 4분할 제출 구조를 적용하였다.

3.2.1 ECS 스웸 시뮬레이션 및 애니메이션

공통 ECS 시뮬레이션은 서로 대립하는 두 진영의 근접 및 원거리 병사 스웸(swarm)으로 구성하였다. 스웸 수준에서는 목표 지점으로의 이동, 대열 유지, 적 탐지 및 교전 상태를 처리하며, 개별 병사는 스웸의 상태와 대열 내 할당 위치를 기준으로 슬롯을 추종하면서 주변 병사와의 분리 및 위치 보정을 수행한다. 이에 따라 각 병사의 위치와 방향이 지속적으로 갱신되는 대규모 동적 장면을 구성하였다.

병사의 행동에 따라 애니메이션 상태도 함께 갱신하며, 위치와 방향, 진영 식별 정보 및 애니메이션 상태를 렌더링에 필요한 인스턴스 상태로 사용하였다. 애니메이션은 두 렌더링 경로에서 공통으로 사용하는 GPU 스키닝 구조에 연결하였다. 이와 같이 ECS에서 생성된 상태는 RMI와 BRG의 공통 입력으로 사용되며, 이후 각 렌더링 경로에서 요구하는 데이터 구조로 구성된다.

3.2.2 RMI 렌더링 경로

RMI 렌더링 경로의 전체 구현 흐름은 그림 3.1과 같다.

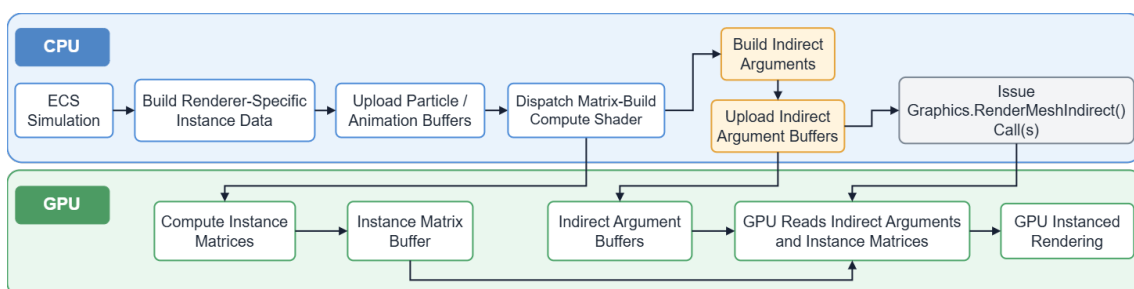


그림 3.1: RMI 렌더링 경로의 구현 흐름

본 구현에서는 ECS에서 생성된 상태를 바탕으로 각 인스턴스의 렌더링용 변환 행렬을 컴퓨터 셰이더에서 생성하여, CPU에서 모든 인스턴스의 최종 변환 행렬을 개별적으로 구성하지 않도록 하였다. 렌더링에 필요한 간접 실행 정보는 활성 서브메시와 제출 대상 인스턴스 범위를 기준으로 구성하여 RMI를 통해 제출한다[7]. 통합 및 4분할 제출은 이러한 기본 데이

터 처리 흐름과 분리하여 인스턴스 범위를 구성하도록 하였으며, 구체적인 제출 구조는 3.2.4 절에서 설명한다.

3.2.3 BRG 렌더링 경로

BRG 렌더링 경로의 전체 구현 흐름은 그림 3.2과 같다.

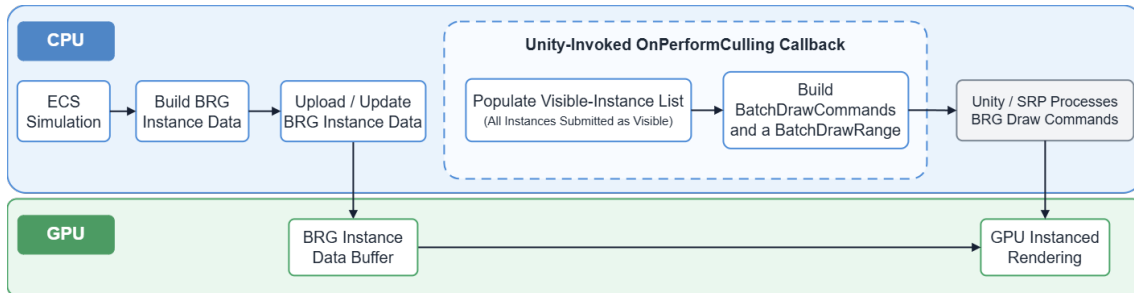


그림 3.2: BRG 렌더링 경로의 구현 흐름

BRG 경로는 RMI와 공통 ECS 상태를 입력으로 사용하되, BRG에서 요구하는 인스턴스 데이터와 렌더링 명령 구조에 맞게 별도로 구성하였다. Unity의 렌더링 절차에서 호출되는 `OnPerformCulling` 콜백에서는 렌더링 경로 비교에 개별 가시성 판정의 차이가 포함되지 않도록 별도의 인스턴스 컬링을 수행하지 않고 전체 인스턴스를 가시 대상으로 제출하였다[5]. 통합 및 4분할 제출은 RMI와 동일한 인스턴스 범위 기준으로 적용하였다.

3.2.4 통합 및 4분할 제출 구조

통합 제출에서는 전체 인스턴스 범위를 하나의 연속된 제출 단위로 구성하고, 4분할 제출에서는 동일한 인스턴스 집합을 크기가 같은 네 개의 연속 범위로 나누어 각각 별도의 제출 단위로 구성하였다. 두 방식에서 전체 인스턴스 수와 렌더링 자원은 변경하지 않으며, 각 제출 단위가 담당하는 인스턴스의 시작 위치와 개수만 달라지도록 하였다.

각 제출 단위는 활성 서브메시에 대응하는 렌더링 명령으로 구성된다. 본 보고서에서는 이와 같이 응용 프로그램 수준에서 구성한 제출 단위를 논리적 제출 명령으로 구분한다. 논리적 제출 명령은 Unity의 렌더링 파이프라인과 그래픽스 백엔드를 거쳐 실제로 관측되는 물리 드로우와 동일한 처리 계층의 값이 아니므로 두 개념을 구분하여 사용한다. 또한 RMI와 BRG는 렌더링 명령을 구성하는 방식 자체가 다르므로 경로 간 논리적 명령 수의 절대값을 직접 대응하기보다, 각 경로 내부에서 통합 제출과 4분할 제출에 따른 구조 변화를 비교한다.

3.3 성능 측정 및 실험 설계

구현한 시스템의 성능 특성을 비교하기 위해 렌더링 경로와 제출 구조를 교차한 반복 실험을 구성하였다. 측정 지표, 부하 조건, 벤치마크 환경 및 자동 측정 절차는 다음과 같이 설정하였다.

3.3.1 측정 지표와 수집 방식

성능 평가는 평균적인 프레임 처리 성능과 프레임 안정성, 처리 단계별 특성 및 렌더링 명령 구조를 함께 확인할 수 있도록 구성하였다. 주요 측정 지표와 수집 및 산출 단위는 표 3.2과 같다.

표 3.2: 성능 측정 지표와 수집 단위

지표	수집, 산출 단위	용도
프레임 처리 시간	매 프레임	평균 및 99번째 백분위 프레임 처리 시간과 프레임 분포 산출
평균 FPS	실행 단위 산출	측정 구간의 완료 프레임 수와 실제 경과시간을 이용한 전체 처리 성능 확인
1% Low FPS	실행 단위 산출	처리 시간이 긴 상위 1% 프레임을 이용한 프레임 안정성 확인
주 스레드 시간	1초 간격 표본	ECS 상태 처리, 렌더링 데이터 구성 및 기타 주 스레드 작업을 포함한 처리 특성 확인
렌더 스레드 시간	1초 간격 표본	Unity 렌더 스레드의 전체 처리 특성 확인
GPU 시간	1초 간격 표본	해당 표본 프레임의 전체 GPU 처리 특성 확인
렌더링 구조 지표	1초 간격 표본	드로우 콜과 경로별 논리적 명령 구조 및 실행 상태 확인

평균 FPS는 측정 구간의 완료 프레임 수를 실제 경과시간으로 나누어 산출하였다. 1% Low FPS는 처리 시간이 긴 상위 1% 프레임의 평균 처리 시간을 FPS로 변환하여 계산하였다.

프레임별 자료와 1초 간격 표본은 수집 대상이 서로 다르므로 동일한 프레임 집합에서 산출된 전체 시간과 부분 시간으로 직접 대응하지 않는다. 또한 주 스레드와 렌더 스레드 시간은 Unity Frame Timing API가 제공하는 각 스레드의 활성 작업 시간이며, GPU 시간은 해당 프레임의 GPU 처리 시간이다[6]. 이 값들은 동일한 프레임을 상호 배타적으로 분할한 구성 요소가 아니므로 단순 합산하지 않는다.

3.3.2 실험 조건과 부하 구성

실험은 렌더링 경로와 제출 구조를 교차한 2×2 요인 설계로 구성하였다. 렌더링 경로는 RMI와 BRG, 제출 구조는 통합 제출과 4분할 제출로 구분하였으며, 네 실험 조건은 표 3.3과 같다.

표 3.3: 렌더링 경로와 제출 구조에 따른 실험 조건

조건	렌더링 경로	제출 구조	비교 목적
E1	RMI	통합 제출	RMI 통합 제출 기준 조건
E2	RMI	4분할 제출	RMI에서 제출 분할에 따른 변화 확인
E3	BRG	통합 제출	통합 제출에서 렌더링 경로 차이 확인
E4	BRG	4분할 제출	BRG에서 제출 분할 및 경로 차이 확인

렌더링 경로와 제출 구조의 효과가 부하에 따라 어떻게 달라지는지 확인하기 위해 총 인스턴스 수 N 과 전체 스왑 수 L 을 부하 조건으로 사용하였다. 구체적인 부하 수준과 반복 실험 조건은 표 3.4과 같다.

표 3.4: 부하 및 반복 실험 조건

항목	수준 및 값
총 인스턴스 수 N	2,000 / 4,000 / 8,000 / 16,000
전체 스왑 수 L	4 / 8 / 16
부하 조합	4수준 $\times$ 3수준 = 12개
워밍업 시간	실행당 3초
측정 시간	실행당 60초
조건별 반복	5회
총 실행 횟수	12개 부하 조합 $\times$ 4조건 $\times$ 5회 = 240회

총 인스턴스는 활성 스왑에 균등하게 배치하며, 스왑당 병사 수는 $k = N/L$ 로 정의하였다. 모든 N 은 각 L 과 제출 분할 수 4로 나누어떨어지도록 설정하여 스왑별 병사 수와 4분할 제출의 인스턴스 범위를 균등하게 유지하였다. 병사 간 분리와 위치 보정은 각 스왑 내부의 병사 쌍을 비교하는 방식으로 구성되어 있으므로, 스왑당 계산량은 $O(k^2)$ 이고 전체 스왑에 대한 계산량은 다음과 같이 나타낼 수 있다.

$$O(Lk^2) = O\left(\frac{N^2}{L}\right) \quad (3.1)$$

따라서 동일한 N 에서도 L 이 감소하면 스윙당 병사 수와 병사 간 상호작용 범위가 함께 증가하여 ECS 작업량이 달라진다. 이에 N 과 L 은 렌더링 경로와 제출 구조의 효과를 서로 다른 부하 조건에서 관찰하기 위한 변수로 사용하였다.

3.3.3 벤치마크 환경 및 통제 조건

모든 실험은 동일한 하드웨어와 소프트웨어 환경에서 수행하였다. 벤치마크에 사용한 주요 환경은 표 3.5과 같다.

표 3.5: 벤치마크 하드웨어 및 소프트웨어 환경

항목	사양
CPU	AMD Ryzen 7 9800X3D, 8 cores / 16 threads, 4.7 GHz
GPU	AMD Radeon RX 9070 XT, 16 GB VRAM, driver 32.0.31021.5001
메모리	32 GB DDR5-5600, 16 GB × 2
운영체제	Windows 11 Education 64-bit, 10.0.26200
Unity	6000.3.9f1
렌더 파이프라인	Universal Render Pipeline (URP) 17.3.0
그래픽스 API	Direct3D 12, feature level 12.2
화면 환경	2560 × 1440 QHD, fullscreen borderless, 180 Hz

렌더링 경로와 제출 구조 이외의 요인이 성능 비교에 미치는 영향을 줄이기 위해 시뮬레이션 상태, 렌더링 자원, 카메라 및 그래픽 설정을 공통으로 유지하였다. 주요 통제 조건은 표 3.6과 같다.

표 3.6: 주요 실험 통제 조건

항목	통제 조건
ECS 시뮬레이션	동일한 상태 갱신 로직, 초기 배치, 난수 시드 및 스윙 구성 사용
렌더링 자원	동일 메시, 텍스처, 애니메이션 및 GPU 스키닝 자원 사용
화면 및 카메라	동일 해상도, 카메라 위치·방향 및 투영 설정 유지
프레임 실행	VSync 비활성화, 목표 프레임률 제한 해제
가시성 및 품질	LOD, 그림자, Motion Vector 및 개별 프리스텀·오클루전 컬링 비활성화, 모든 인스턴스를 가시 대상으로 유지
비교 외 동적 객체	GameObject 기반 투사체 생성 및 시각화 비활성화
GPU 버퍼	최대 실험 부하를 수용하도록 사전 확보하고 측정 중 크기 재할당 방지
렌더러 활성화	해당 실험 조건에서 사용하는 렌더링 경로만 활성화

이러한 통제는 가시성 판정이나 품질 조절에 따른 실제 렌더링 작업량의 변화가 비교 결과에 포함되지 않도록 하고, 동일한 전체 가시 조건에서 렌더링 경로와 제출 구조의 차이를 비교하기 위한 것이다.

3.3.4 자동 측정 및 분석 절차

반복 측정의 조건을 일관되게 유지하기 위해 실험 설정, 실행 전 검증, 측정 및 결과 저장 과정을 자동화하였다. 자동 성능 측정 도구의 실행 설정 화면은 그림 3.3과 같다.

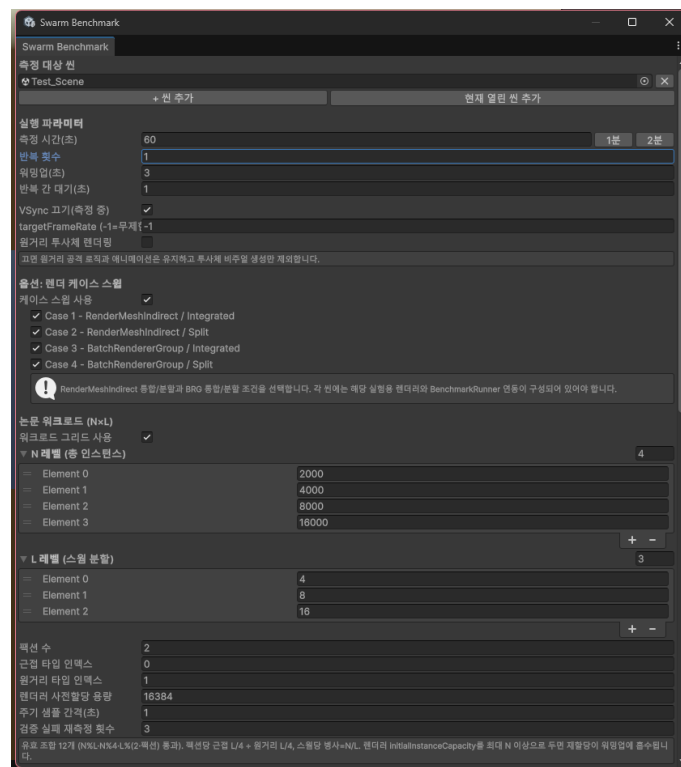


그림 3.3: 자동 성능 측정 도구의 실행 설정 화면

측정 도구는 실행 전에 활성 렌더러와 GPU 버퍼 용량, 인스턴스 수 및 예상 명령 구조가 설정한 실험 조건과 일치하는지 확인하도록 구성하였다. 각 실행에서는 ECS 엔티티와 렌더링 자원을 초기화한 뒤 위밍업을 수행하고, 위밍업 구간의 값은 결과에서 제외하였다. 이후 측정값은 파일 기록에 따른 추가 부하를 줄이기 위해 메모리에 우선 저장하고, 실행 종료 후 실행 단위 요약값과 실험 조건 및 검증 정보를 CSV 파일로 일괄 저장하였다. 설정 검증에 실패하거나 측정값이 누락된 실행은 결과에서 제외하고 동일한 조건으로 다시 수행하였다.

분석의 독립 단위는 한 번의 측정 실행으로 정의하였다. 한 실행 안에서 수집된 프레임별 자료와 1초 간격 표본은 동일한 실행에 속한 반복 관측값으로 취급하고 각각을 별도의 독립 실험으로 간주하지 않았다. 조건별 비교는 각 실행에서 산출한 실행 단위 요약값을 기준으로 수행하였다.

제4장 연구 결과 분석 및 평가

4.1 프로젝트 수행 결과

본 프로젝트는 시스템 설계와 구현, 실험 구조 재설계, 반복 벤치마크 및 결과 분석의 순서로 진행하였다. 주요 개발 일정과 구성원별 역할은 다음과 같이 정리하였다.

4.1.1 개발 일정

프로젝트는 ECS 기반 시뮬레이션과 렌더링 시스템 구현을 시작으로 성능 측정 환경을 구축한 뒤, 비교 구조를 재설계하여 최종 실험 환경을 완성하는 순서로 진행하였다. 이후 반복 벤치마크와 결과 분석 및 논문화 작업을 수행하고, 추가 검증과 최종보고서 작성으로 프로젝트를 마무리하였다. 세부 일정은 표 4.1과 같다.

4.1.2 구성원 역할 분담

구성원은 렌더링 시스템, 군집·애니메이션 시스템 및 성능 벤치마크를 중심으로 주요 구현 영역을 분담하였으며, 최종 실험 단계에서는 데이터 수집과 분석 및 논문 작성에 필요한 작업을 함께 수행하였다. 세부 역할은 표 4.2과 같다.

4.2 성능 실험 결과 및 분석

제3장에서 설정한 전체 본 측정을 수행하고, 수집된 결과를 바탕으로 렌더링 경로와 제출 구조, 처리 단계 및 부하 조건에 따른 성능 특성을 분석하였다. 성능 측정 중의 대표적인 실험 장면은 그림 4.1과 같다.

전체 결과 가운데 주요 성능 결과는 표 4.3과 같으며, 이후 렌더링 경로와 제출 구조, 처리 단계 및 부하 조건에 따라 분석하였다.

표 4.1: 개발 일정표

개발구분	세부항목	3	4	5	6	7	8	9
기획 및 설계	연구 주제 선정 및 관련 연구 조사	○	○					
기본 시스템 구현	ECS 군집 시뮬레이션 및 GPU 스키닝 구현		○	○				
렌더링 시스템 구현	BRG 기반 렌더링 구조 및 인스턴스 데이터 처리 구현			○	○			
성능 측정 환경 구축	성능 지표 수집 및 자동 벤치마크 기능 구현				○	○		
실험 구조 재설계	최종 비교 구조로 변경 (7월 13일)					○		
최종 시스템 보완	RMI 경로 및 통합, 4분할 제출 구조 구현 (7월 13-15일)					○		
성능 실험	최종 반복 벤치마크 수행 (7월 16일)					○		
결과 분석 및 논문화	결과 분석, 시각화 및 논문 작성 (7월 17-29일)					○		
	학회 논문 제출 (7월 30일)					○		
최종 정리	추가 검증 및 최종보고서, 졸업과제 산출물 정리						○	○

표 4.2: 역할 분담

학번	성명	역할
202255658	김재식	ECS 상태 및 인스턴스 데이터 구조 설계, 구현 BRG 렌더러 및 데이터 지향 인스턴싱 셰이더 구현 RMI 렌더링 경로 및 통합, 4분할 제출 구조 구현, 개선 관련 연구 조사 및 실험 설계 논문 작성 및 프로젝트 총괄
202155593	이천서	GPU 스키닝 휴머노이드 애니메이션 상태 입력 및 위상 분산 구현 원거리 유닛, 투사체, 팩션 표현 등 유형별 유닛 구성 스웬 스폰 및 베이스 자동화 구현 군집, 애니메이션 상태 전환 검증 최종 실험 결과 데이터 분석
202255568	안지홍	성능 지표 자동 수집 벤치마크 도구 구현 CSV 저장 및 빌드 자동화 기능 구현 휴머노이드 프리랩, 3D 에셋의 GPU 스키닝 렌더링 연결 최종 비교 조건 반복 벤치마크 수행 및 성능 데이터 수집 결과 데이터 분석, 시각화 및 성능 특성 해석

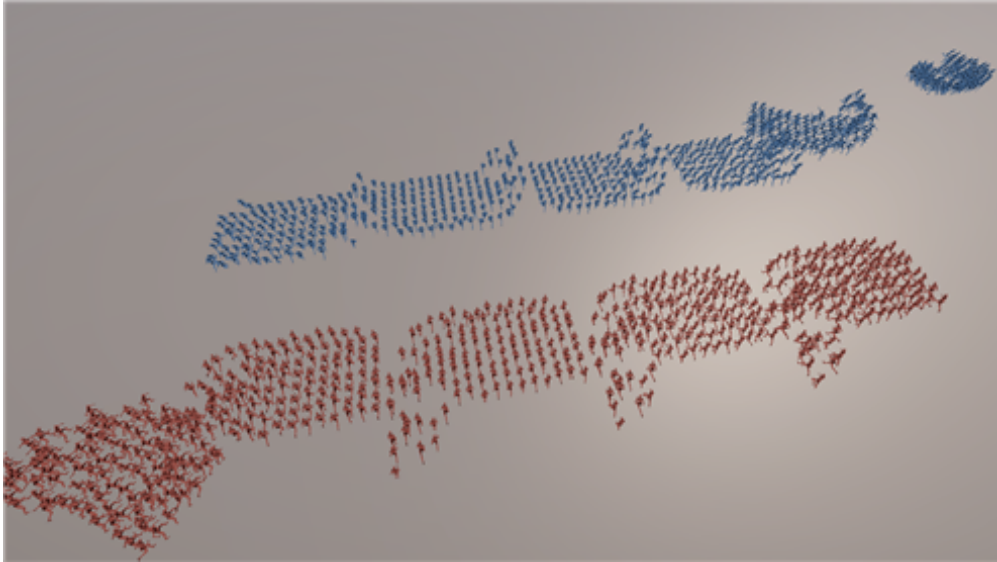


그림 4.1: 성능 측정 중 대표적인 실험 실행 장면

표 4.3: 주요 성능 실험 결과 요약

비교 항목	대표 조건	주요 결과
렌더링 경로	$L = 16$, 통합 제출	BRG의 평균 프레임 처리 시간이 RMI보다 6.3~8.5% 높게 나타남
제출 구조	$L = 16$	통합 제출과 4분할 제출의 평균 프레임 처리 시간 차이가 두 경로에서 모두 0.5% 이내로 나타남
렌더링 제한 내 중간적 특성	$N = 8,000$, $L = 4$, E1/E3	E1은 16.41 ms, 60.95 FPS, 1% Low 43.18 FPS, E3는 20.86 ms, 48.18 FPS, 1% Low 30.75 FPS
L 감소에 따른 시뮬레이션 부하	$N = 16,000$, E1	$L = 16, 8, 4$ 에서 평균 프레임 처리 시간이 각각 31.73 ms, 378.42 ms, 847.52 ms로 증가

4.2.1 렌더링 경로 및 제출 구조의 성능 비교

$L = 16$ 에서는 인스턴스 수가 증가할수록 RMI와 BRG의 평균 프레임 처리 시간 차이가 확대되는 경향이 나타났으며, 두 제출 구조 모두에서 BRG가 더 높은 값을 보였다. 결과는 그림 4.2과 같다.

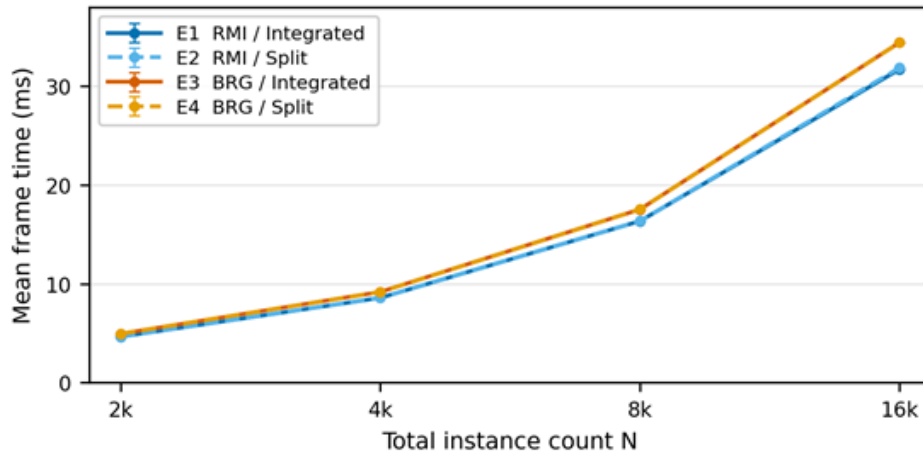


그림 4.2: $L = 16$ 에서 인스턴스 수에 따른 평균 프레임 처리 시간

반면 각 렌더링 경로에서 통합 제출과 4분할 제출을 비교하면 인스턴스 수 증가에 따른 일관된 개선 또는 악화 방향은 확인되지 않았다. 따라서 동일한 인스턴스 집합을 여러 제출 단위로 나누는 것만으로는 본 실험 범위에서 뚜렷한 프레임 성능 개선이 나타나지 않았다.

Unity에서 관측한 전체 드로우 수는 RMI 조건에서는 인스턴스 수에 따른 변화가 크지 않았으나, BRG 조건에서는 인스턴스 수와 함께 증가하였다. 결과는 그림 4.3과 같다.

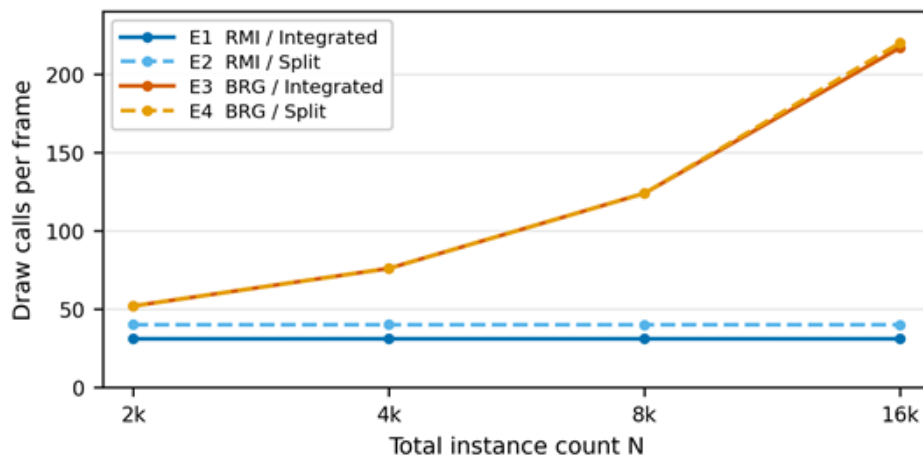


그림 4.3: $L = 16$ 에서 인스턴스 수에 따른 Unity 관측 드로우 콜 수

다만 관측 드로우 수는 논리적 제출 명령과 동일한 처리 계층의 지표가 아니며, 그 변화도 프레임 성능과 직접 대응하지 않았다.

4.2.2 단계별 처리 시간과 해석

렌더링 경로에 따른 차이가 처리 단계별 측정값에서 어떻게 나타나는지 확인하기 위해 $L = 16$ 의 통합 제출 조건인 E1과 E3의 주 스레드, 렌더 스레드 및 GPU 시간을 비교하였다. 결과는 그림 4.4과 같다.

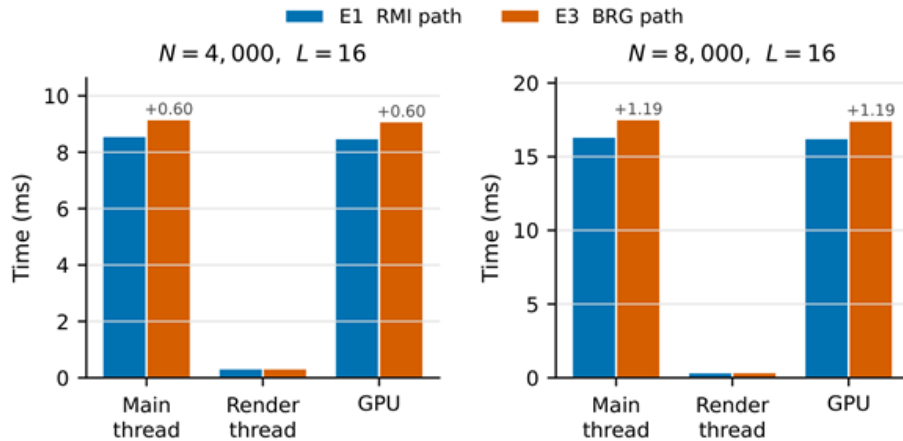


그림 4.4: $L = 16$ 통합 제출 조건의 주 스레드, 렌더 스레드 및 GPU 시간

렌더 스레드 시간은 약 0.32–0.37 ms 범위에서 변화가 작았으며, 렌더링 경로에 따른 차이는 주 스레드와 GPU 시간에서 함께 나타났다. $N = 16,000$ 에서 E1의 주 스레드와 GPU 시간은 각각 31.72 ms와 31.61 ms였고, E3에서는 각각 34.42 ms와 34.33 ms였다. 이는 동일한 부하에서 관측된 평균 프레임 처리 시간의 경로별 차이와 같은 방향의 변화이다.

단계별 시간은 서로 중첩될 수 있고 각 단계의 공통 시스템 처리도 포함하므로 전체 프레임 처리 시간의 구성 요소로 단순 합산하지 않았다. 또한 관측된 차이를 특정 BRG 내부 작업의 비용으로 귀속하지 않고, 동일한 ECS 조건에서 서로 다른 렌더링 경로를 적용했을 때 나타난 전체 처리 특성의 차이로 해석하였다.

4.2.3 부하 조건에 따른 성능 변화

렌더링 경로와 제출 구조의 상대적인 영향은 인스턴스 수와 스왑 구성에 따라 달라졌다. 그림 4.5은 E1에서 총 인스턴스 수 N 과 전체 스왑 수 L 의 조합에 따른 평균 프레임 처리 시간을 나타낸다.

$L = 16$ 은 앞선 렌더링 경로 비교에서 경로 간 성능 차이가 비교적 명확하게 관측된 부하 영역인 반면, 동일한 N 에서 L 이 감소하면 평균 프레임 처리 시간이 빠르게 증가하였다. 이는 제3장에서 설명한 스왑 내부 병사 쌍 처리 구조에서 L 감소에 따라 스왑당 병사 수가 증가하고, 공통 ECS 시뮬레이션의 처리 비용이 증가하는 특성과 일치한다.

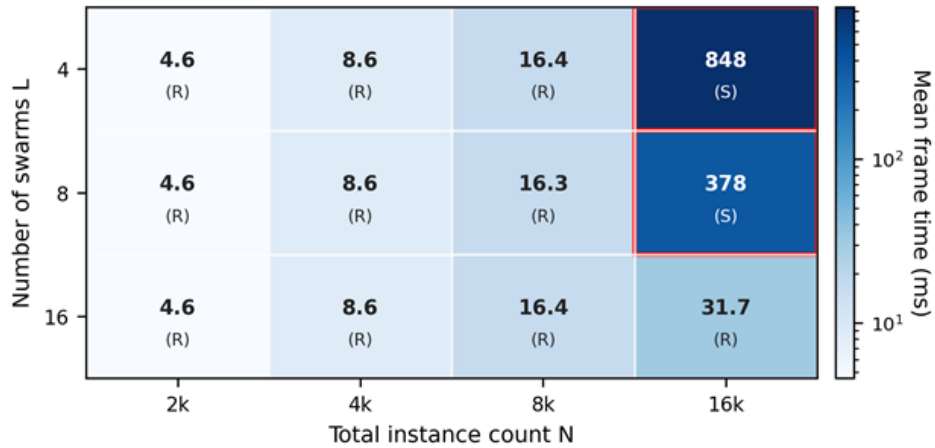


그림 4.5: E1의 $N \times L$ 부하 조합별 평균 프레임 처리 시간 (R: rendering-limited; S: simulation-limited)

렌더링 제한으로 분류되지만 중간적 특성을 보이는 $N = 8,000$, $L = 4$ 에서는 프레임 처리 시간과 평균 FPS, 1% Low FPS에서 렌더링 경로 간 차이가 관측되었다.

반면 $N = 16,000$ 에서 $L = 8$ 과 $L = 4$ 로 구성된 고부하 조건에서는 네 실행 조건 모두 큰 프레임 처리 시간을 보였다. 이러한 조건에서는 공통 ECS 시뮬레이션 비용의 영향이 전체 프레임 성능에서 크게 나타나면서 렌더링 경로와 제출 구조에 따른 상대적인 차이가 가려졌다.

이상의 결과에서 렌더링 경로의 성능 차이는 렌더링 부하의 영향이 상대적으로 큰 조건에서 명확하게 나타났으며, 단순한 제출 분할의 효과는 제한적이었다. 반면 공통 ECS 시뮬레이션 부하가 증가한 조건에서는 해당 비용이 전체 프레임 성능을 지배하면서 렌더링 경로와 제출 구조의 차이가 상대적으로 가려졌다. 따라서 렌더링 경로와 제출 구조에 관한 비교 결과는 렌더링 제한 구간을 중심으로 해석한다.

4.3 연구 성과

본 프로젝트와 동일한 연구 주제로 작성한 논문 「Unity ECS 대규모 인스턴스 렌더링의 간접 드로우와 엔진 관리 배치 제출 성능 비교」가 2026년 8월 한국연구재단 등재학술지인 멀티미디어학회논문지 제29권 제8호, pp. 1220–1233에 게재되었다.

제5장 결론 및 향후 연구 방향

5.1 결론 및 기대효과

본 프로젝트에서는 Unity ECS 기반 대규모 동적 3D 인스턴스 시스템을 구축하고, 공통 시뮬레이션과 렌더링 조건을 통제된 환경에서 RMI와 BRG의 두 렌더링 경로에 통합 제출과 4분할 제출을 교차 적용하여 성능 특성을 비교하였다. 실험 결과 렌더링 경로와 제출 구조의 영향은 부하 조건에 따라 다르게 나타났으며, 실제 작업량 감소를 수반하지 않는 단순한 제출 분할의 효과는 제한적이었다. 또한 공통 ECS 시뮬레이션 비용이 증가하여 전체 프레임 성능을 지배하는 조건에서는 렌더링 경로와 제출 구조에 따른 차이가 상대적으로 가려졌다. 따라서 대규모 동적 인스턴스 시스템의 성능은 특정 렌더링 방식이나 제출 구조의 일률적인 우위보다 실제 부하에서 지배적으로 작용하는 처리 특성과 함께 판단할 필요가 있다.

본 프로젝트의 의의는 특정 렌더링 경로의 보편적인 우위를 제시하는 데 있지 않고, 공통 시뮬레이션 및 렌더링 조건을 통제된 비교 환경에서 렌더링 경로와 제출 구조의 영향을 구분하여 분석함으로써 부하 조건에 따른 설계 판단의 근거를 마련한 데 있다. 모든 인스턴스를 매 프레임 갱신하고 가시화하는 조건에서는 RMI를 성능 중심의 우선 후보로 고려할 수 있으며, 제출 단위별 컬링이나 부분 갱신으로 실제 처리해야 하는 작업량을 줄일 수 없는 경우에는 통합 제출을 기본 구조로 고려할 수 있다. 반대로 객체 간 상호작용과 상태 갱신 비용이 전체 프레임을 지배하는 조건에서는 렌더링 구조를 변경하기보다 시뮬레이션의 상태 갱신과 탐색 구조를 우선적으로 최적화하는 것이 적절하다. 이러한 결과는 Unity ECS 기반 대규모 동적 장면에서 렌더링 경로와 제출 구조를 선택하고 시스템의 최적화 우선순위를 판단하기 위한 실증적 기준으로 활용될 수 있다.

5.2 향후 연구 방향

본 프로젝트의 실험은 렌더링 경로와 제출 구조 자체의 영향을 구분하기 위해 컬링, 세부 수준 조절(LOD) 및 그림자를 제외하고 모든 인스턴스를 지속적으로 갱신, 가시화하는 조건으로 구성하였다. 또한 단일 하드웨어, 소프트웨어 환경에서 통합 제출과 하나의 4분할 제출

조건을 비교하였으므로, 본 연구의 결과는 전체 가시, 전체 갱신, 동일 자원 조건에서의 기준 성능으로 한정하여 해석할 필요가 있다. 따라서 실제 게임 장면과 같이 가시성, 거리 및 객체 구성에 따라 처리해야 하는 렌더링 작업량이 달라지는 조건에서 렌더링 경로와 제출 구조의 성능 특성을 추가로 검증할 필요가 있다.

향후에는 컬링과 그림자, LOD, 부분 갱신 및 다중 메시, 머티리얼 조건을 단계적으로 적용하여 실제 렌더링 작업량이 변화하는 환경으로 실험 범위를 확장할 수 있다. 또한 현재의 통합 및 4분할 제출에서 나아가 제출 입도를 여러 수준으로 구성하고, 각 제출 단위에서 컬링이나 부분 갱신을 통해 감소하는 실제 작업량과 제출 단위를 세분화하면서 증가하는 관리 비용을 함께 비교할 필요가 있다. 이를 통해 제출 단위의 세분화가 실제 처리량 감소로 이어지는 조건과 그렇지 않은 조건을 구분하고, 대규모 동적 인스턴스 장면의 부하 특성에 따라 적절한 렌더링 경로와 제출 구조를 선택하기 위한 기준을 보다 구체화할 수 있을 것이다.

참고문헌

- [1] A. Beacco, N. Pelechano, and C. Andújar. A survey of real-time crowd rendering. *Computer Graphics Forum*, 35(8):32–50, 2016.
- [2] Y. Dong and C. Peng. Real-time large crowd rendering with efficient character and instance management on GPU. *International Journal of Computer Games Technology*, 2019:1–15, 2019. Article ID 1792304.
- [3] Brennan Shacklett, Erik Wijmans, Aleksei Petrenko, Manolis Savva, Dhruv Batra, Vladlen Koltun, and Kayvon Fatahalian. Large batch simulation for deep reinforcement learning. In *Proceedings of the 9th International Conference on Learning Representations (ICLR)*, pages 1–17, 2021.
- [4] M. Sung. Study on entity-component system framework benchmarking for game development. *Journal of Korea Multimedia Society*, 28(1):58–65, 2025.
- [5] Unity Technologies. BatchRendererGroup API in URP. <https://docs.unity3d.com/6000.3/Documentation/Manual/batch-renderer-group.html>. Checked on 2026-07-17.
- [6] Unity Technologies. Frame Timing API Counter Reference. <https://docs.unity3d.com/6000.3/Documentation/Manual/frame-timing-manager-counter-reference.html>. Checked on 2026-09-07.
- [7] Unity Technologies. Graphics.RenderMeshIndirect. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Graphics.RenderMeshIndirect.html>. Checked on 2026-07-17.
- [8] L. Voisard, H. De Freitas Serra, C. Politowski, F. Petrillo, and Y.-G. Guéhéneuc. A mapping study of the entity component system pattern. In *Proceedings of the 2025 IEEE/ACM 9th International Workshop on Games and Software Engineering (GAS)*, pages 33–40, 2025.

- [9] David Wingqvist, Filip Wickström, and Suejb Memeti. Evaluating the performance of object-oriented and data-oriented design with multi-threading in game development. In *Proceedings of the 2022 IEEE Games, Entertainment, Media Conference (GEM)*, pages 1–6, 2022.